TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TỔNG HỢP TOÀN BỘ ĐỒ ÁN OOP (ĐIỂM 10/10)

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (JAVA OOP) • BẢN TỔNG HỢP ĐẦY ĐỦ

Toàn bộ 4 Thành viên nhóm | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

HƯỚNG DẪN HỌC & BẢO VỆ VẤN ĐÁP ĐỒ ÁN OOP (ĐIỂM 10/10)

> **Phân công thành viên:** Thành viên 1

> **Các lớp phụ trách (8 lớp & interface):** `SanPhamThoiTrang` , `AoKhoac` , `AoThun` , `QuanJean` , `PhuKien` , `VayDam` , `GiayDep` , `IBaoHanh` , `IGiamGia` .

> **Nhiệm vụ cốt lõi:** Nắm vững cấu trúc cây kế thừa sản phẩm, lớp trừu tượng, phương thức trừu tượng, tính đóng gói dữ liệu và cơ chế đa hình bảo hành qua interface.

1. TỔNG QUAN CÂY PHÂN CẤP SẢN PHẨM THỜI TRANG

```
<<Interface>> IGiamGia
                            ^
                            | implements
                [SanPhamThoiTrang] (abstract)
                /   |     |    \     \      \
               /    |     |     \     \      \
      [AoKhoac] [AoThun] [QuanJean] [PhuKien] [VayDam] [GiayDep]
          |                              |                  |
          +------------ implements ----+------------------+
                                       v
                              <<Interface>> IBaoHanh
```

- **Mối quan hệ IS-A (Là một):** Áo khoác, Áo thun, Quần jean, Phụ kiện, Váy đầm, Giày dép đều là một Sản phẩm thời trang (`extends SanPhamThoiTrang`).
- **Mối quan hệ CAN-DO (Có khả năng):** Áo khoác, Phụ kiện, Giày dép có thể bảo hành (`implements IBaoHanh`). Quần áo thông thường không có bảo hành.

2. GIẢI THÍCH CHI TIẾT TỪNG ĐOẠN CODE CỐT LÕI

Đoạn 1: Khai báo lớp trừu tượng `SanPhamThoiTrang`

```
public abstract class SanPhamThoiTrang implements IGiamGia {
    public static final double TI_LE_VAT = 0.10;
    private static int tongSoLuongSanPhamDaTao = 0;

    private String Type;
    private String MaSP;
    private String TenSP;
    private double GiaSP;
```

- **Giải thích từng dòng code:**

- `public abstract class SanPhamThoiTrang` : Khai báo lớp trừu tượng. Lớp này đóng vai trò lớp cha định hình khuôn mẫu cho mọi sản phẩm, **không thể khởi tạo trực tiếp** bằng lệnh `new SanPhamThoiTrang()` mà chỉ dùng làm tham chiếu đa hình.
- `implements IGiamGia` : Cam kết hiện thực interface giảm giá sản phẩm.
- `public static final double TI_LE_VAT = 0.10` : Thuộc tính static hằng số (10% thuế VAT). Dùng chung cho toàn bộ sản phẩm trong cửa hàng, không tốn bộ nhớ cấp phát riêng cho từng đối tượng.
- `private static int tongSoLuongSanPhamDaTao = 0` : Biến static đếm tổng số sản phẩm đã được tạo trong bộ nhớ.
- `private String Type, MaSP, TenSP; private double GiaSP;` : Tính **Đóng gói (Encapsulation)**. Toàn bộ thuộc tính cơ sở được đặt `private` để bảo vệ dữ liệu, chống việc gán giá trị bừa bãi từ bên ngoài.

Đoạn 2: Hàm khởi tạo (Constructor) có tham số & Ràng buộc dữ liệu

```
public SanPhamThoiTrang(String Type, String MaSP, String TenSP, double GiaSP) {
        this.Type = (Type != null) ? Type : "ChuaPhanLoai";
        this.MaSP = (MaSP != null) ? MaSP : "";
        this.TenSP = (TenSP != null) ? TenSP : "";
        this.GiaSP = (GiaSP >= 0) ? GiaSP : 0;
        tongSoLuongSanPhamDaTao++;
    }
```

- **Giải thích từng dòng code:**
- `this.Type = (Type != null) ? Type : "ChuaPhanLoai"` : Kiểm tra null an toàn. Tránh lỗi `NullPointerException` khi thao tác chuỗi.
- `this.GiaSP = (GiaSP >= 0) ? GiaSP : 0` : Ràng buộc giá tiền không được âm. Nếu truyền số âm sẽ tự động đưa về 0.
- `tongSoLuongSanPhamDaTao++` : Mỗi khi từ khóa `new` được gọi ở bất kỳ lớp con nào (`AoKhoac` , `AoThun` ...), biến static này tự động tăng 1 để quản lý số lượng đối tượng đã sinh ra.

Đoạn 3: Các phương thức trừu tượng (Abstract Methods)

```
public abstract String getStringSaveFile();
    public abstract String getThongTinRieng();
```

- **Giải thích từng dòng code:**
- Phương thức trừu tượng chỉ có phần khai báo, **không có thân hàm `{}`**, kết thúc bằng dấu chấm phẩy `;` .
- Ép buộc 100% các lớp con kế thừa (`AoKhoac` , `AoThun` , `QuanJean` ...) **bắt buộc phải ghi đè (@Override)** để định nghĩa cách lấy thông tin riêng và chuỗi lưu file phù hợp với đặc thù từng loại mặt hàng.

Đoạn 4: Tính toán giá sau giảm & Thuế VAT

```
public double VATcalculation() {
        return this.GiaSP * TI_LE_VAT;
    }

    @Override
    public double calculateAfterDiscount(double discountPercentage) {
```

```
        return this.GiaSP - (this.GiaSP * discountPercentage / 100);
    }
```

- **Giải thích từng dòng code:**
- `VATcalculation()` : Tính tiền thuế GTGT bằng cách lấy đơn giá nhân với hằng số `TI_LE_VAT` (10%).
- `calculateAfterDiscount(...)` : Hiện thực từ Interface `IGiamGia` . Công thức: `Giá sau giảm = Giá gốc - (Giá gốc * % giảm / 100)` . Cung cấp khả năng tính giá linh hoạt cho các đợt khuyến mãi.

Đoạn 5: Setter bảo vệ dữ liệu & Chống lỗi định dạng file CSV

```
public void setTenSP(String TenSP) {
        if (TenSP != null && !TenSP.trim().isEmpty()) {
            this.TenSP = TenSP.trim().replace(",", " ");
        }
    }

    public boolean setGiaBan(double GiaBanMoi) {
        if (GiaBanMoi >= 0) {
            this.GiaSP = GiaBanMoi;
            return true;
        } else {
            System.out.println("Gia ban phai lon hon hoac bang 0!");
            return false;
        }
    }
```

- **Giải thích từng dòng code:**
- `replace(",", " ")` : Do hệ thống lưu file `products.txt` dạng dòng phân tách bằng dấu phẩy (CSV format), nếu tên sản phẩm chứa dấu phẩy (ví dụ: "Áo gió, nam, cao cấp") thì khi đọc file bằng hàm `split(",")` sẽ làm sai số lượng cột. Thao tác này khử dấu phẩy triệt để để bảo vệ tính toàn vẹn của file dữ liệu.
- `setGiaBan` : Trả về `boolean` báo hiệu thành công hay thất bại. Chặn không cho cập nhật giá âm.

Đoạn 6: Lớp con `AoKhoac` kế thừa & Hiện thực `IBaoHanh`

```
public class AoKhoac extends SanPhamThoiTrang implements IBaoHanh {
    private boolean hasHat;

    public AoKhoac(String Type, String MaSP, String TenSP, double GiaSP, boolean hasHat) {
        super(Type, MaSP, TenSP, GiaSP);
        this.hasHat = hasHat;
    }

    @Override
    public String getThongTinRieng() {
        return "Co non: " + (this.hasHat ? "Co" : "Khong");
    }

    @Override
    public void printWarrantyCard() {
        System.out.println("+------------------------------------------------------------+");
        System.out.println("|                    PHIEU BAO HANH                          |");
        System.out.println("+------------------------------------------------------------+");
        System.out.printf("| Ten san pham      : %-33s |\n", getTenSP());
        System.out.printf("| Ma san pham       : %-33s |\n", getMaSP());
```

```
        System.out.printf("| Thoi han bao hanh : %-33s |\n", getWarrantyPeriod() + " thang");
        System.out.println("+-------------------------------------------------------+");
    }

    @Override
    public int getWarrantyPeriod() {
        return 12;
    }

    @Override
    public String getStringSaveFile() {
        return getType() + "," + getMaSP() + "," + getTenSP() + "," + getGiaBan() + "," +
this.hasHat;
    }
}
```

- **Giải thích từng dòng code:**
- `extends SanPhamThoiTrang` : Kế thừa toàn bộ mã nguồn quản lý mã, tên, giá, VAT từ lớp cha.
- `implements IBaoHanh` : Khẳng định Áo khoác có chế độ bảo hành chính hãng.
- `super(Type, MaSP, TenSP, GiaSP)` : Lời gọi constructor của lớp cha để khởi tạo các thuộc tính chung. Phải nằm ở **dòng đầu tiên** của constructor lớp con.
- `this.hasHat = hasHat` : Khởi tạo thuộc tính riêng của áo khoác (có mũ nón hay không).
- `@Override getThongTinRieng()` : Hiện thực phương thức trừu tượng, trả về chuỗi thông tin riêng hiển thị trên bảng Console.
- `@Override getWarrantyPeriod()` : Trả về thời hạn bảo hành cố định là **12 tháng**.
- `@Override printWarrantyCard()` : Đóng khung in phiếu bảo hành chuẩn format.
- `@Override getStringSaveFile()` : Xuất định dạng 5 cột chuẩn để ghi xuống `products.txt` .

Đoạn 7: Các lớp con chuyên biệt khác

1. **`AoThun`**:
- Thuộc tính riêng: `private String size;`
- Logic chuẩn hóa: Khi khởi tạo hoặc gọi `setSize(String sz)` , tự động chuyển thành chữ hoa `S, M, L, XL` . Nếu nhập sai mặc định gán là `"M"` .
2. **`QuanJean`**:
- Thuộc tính riêng: `private String Style;` (Kiểu dáng: Skinny, Baggy, Ống rộng). Có xử lý lọc bỏ dấu phẩy.
3. **`PhuKien`**:
- Thuộc tính riêng: `private String Material;` (Chất liệu: Da bò, Kim loại...).
- Hiện thực `IBaoHanh` với thời hạn bảo hành **6 tháng**.
4. **`VayDam`**:
- Thuộc tính riêng: `private String chatLieuVai;` (Voan, Lụa, Cotton...).
5. **`GiayDep`**:
- Thuộc tính riêng: `private int sizeGiay;`
- Ràng buộc dữ liệu: Size giày nằm trong khoảng hợp lệ từ **35 đến 45**.
- Hiện thực `IBaoHanh` với thời hạn bảo hành **3 tháng**.

3. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU & CÁCH TRẢ LỜI ĐẠT 10 ĐIỂM

CÂU HỎI VẤN ĐÁP: Tại sao `SanPhamThoiTrang` lại là Abstract Class mà không phải Interface?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, `SanPhamThoiTrang` chứa các thuộc tính trạng thái cụ thể (`Type`, `MaSP`, `TenSP`, `GiaSP`) và các phương thức đã có thân hàm cài đặt sẵn (`VATcalculation`, `formatTien`, `setGiaBan`). Interface không thể lưu trạng thái đối tượng như vậy. Ngoài ra, quan hệ giữa Áo khoác/Áo thun với Sản phẩm là quan hệ bản chất kế thừa **IS-A** (Áo khoác là một sản phẩm), do đó thiết kế Lớp trừu tượng là hoàn toàn chuẩn xác theo nguyên lý OOP.

CÂU HỎI VẤN ĐÁP: Từ khóa `super` trong constructor lớp con dùng để làm gì? Bỏ đi có được không?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, từ khóa `super(...)` dùng để gọi hàm thiết lập của lớp cha nhằm khởi tạo các thuộc tính `Type`, `MaSP`, `TenSP`, `GiaSP` do lớp cha quản lý. Lệnh này bắt buộc phải nằm ở dòng đầu tiên trong constructor của lớp con. Nếu không ghi tường minh `super(...)` có tham số, Java sẽ tự động gọi `super()` mặc định không tham số của lớp cha.

CÂU HỎI VẤN ĐÁP: Kỹ thuật Đa hình (Polymorphism) thể hiện như thế nào trong phần quản lý sản phẩm này?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, đa hình thể hiện rõ rệt ở 2 điểm:

1. **Đa hình qua lớp trừu tượng:** Khi một biến tham chiếu kiểu cha `SanPhamThoiTrang sp` duyệt qua danh sách, gọi `sp.getThongTinRieng()` hay `sp.getStringSaveFile()`, chương trình sẽ tự động kích hoạt phiên bản phương thức tương ứng của từng đối tượng thực tế tại runtime (`AoKhoac`, `AoThun`, `GiayDep`...).
2. **Đa hình qua Interface `IBaoHanh`:** Trong lớp hóa đơn, chương trình kiểm tra `if (sp instanceof IBaoHanh) { ((IBaoHanh) sp).printWarrantyCard(); }`. Dù cùng là giao diện bảo hành, Áo khoác sẽ in 12 tháng, Phụ kiện in 6 tháng và Giày dép in 3 tháng.

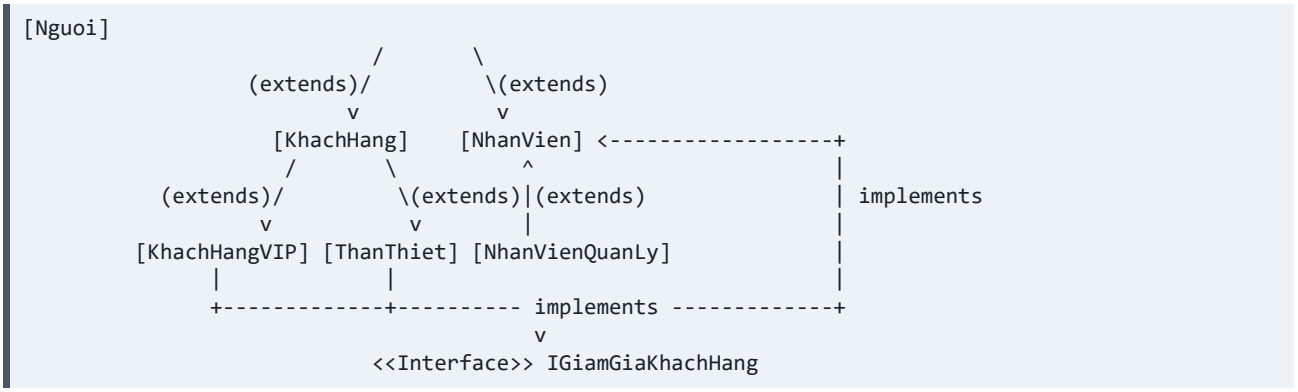
HƯỚNG DẪN HỌC & BẢO VỆ VẤN ĐÁP ĐỒ ÁN OOP (ĐIỂM 10/10)

> **Phân công thành viên:** Thành viên 2

> **Các lớp phụ trách (8 lớp & interface):** `Nguoi` , `KhachHang` , `KhachHangVIP` , `KhachHangThanThiet` , `NhanVien` , `NhanVienQuanLy` , `NhaCungCap` , `IGiamGiaKhachHang` .

> **Nhiệm vụ cốt lõi:** Nắm vững cấu trúc phân cấp kế thừa con người, cơ chế chiết khấu đa hình thông qua interface `IGiamGiaKhachHang` , tính năng quản lý đối tác nhà cung cấp và bảo vệ dữ liệu người dùng.

1. CÂY PHÂN CẤP KẾ THỪA CON NGƯỜI (HUMAN HIERARCHY)



- **Lớp gốc `Nguoi`**: Nắm giữ các thuộc tính định danh cá nhân căn bản nhất.
- **Nhánh `KhachHang`**: Quản lý các phân khúc khách hàng mua sắm tại cửa hàng.
- **Nhánh `NhanVien`**: Quản lý nhân sự làm việc tại cửa hàng, hưởng chính sách mua hàng nội bộ.
- **Đa hình chiết khấu qua Interface `IGiamGiaKhachHang`**: Thống nhất phương thức `applyDiscount(double)` cho các đối tượng được hưởng ưu đãi giảm giá.

2. GIẢI THÍCH CHI TIẾT TỪNG ĐOẠN CODE CỐT LÕI

Đoạn 1: Lớp cơ sở `Nguoi` & Tính Đóng gói (Encapsulation)

```
public class Nguoi {
    private String name;
    private String numberPhone;

    public Nguoi() {
        this.name = "";
        this.numberPhone = "";
    }

    public Nguoi(String name, String numberPhone) {
        this.name = (name != null) ? name.trim().replace(",", " ") : "";
        this.numberPhone = (numberPhone != null) ? numberPhone.trim().replace(",", "") : "";
    }

    public String getName() { return name; }
    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name.trim().replace(",", " ");
        }
```

```
    }

    public String getNumberPhone() { return numberPhone; }
    public void setNumberPhone(String numberPhone) {
        if (numberPhone != null) {
            this.numberPhone = numberPhone.trim().replace(",", "");
        }
    }
}
```

- **Giải thích từng dòng code:**
- `private String name; private String numberPhone;` : Thuộc tính được bảo vệ chặt chẽ, không cho phép truy cập trực tiếp từ bên ngoài lớp.
- `name.trim().replace(",", " ")` : Cắt bỏ khoảng trắng thừa ở 2 đầu chuỗi và thay thế dấu phẩy bằng khoảng trắng. Giữ an toàn cấu trúc khi lưu xuống file `users.txt` (định dạng CSV).
- `numberPhone.trim().replace(",", "")` : Chuẩn hóa số điện thoại, loại bỏ các ký tự dấu phẩy vô tình gõ nhầm.
- Các hàm `setName` và `setNumberPhone` kiểm tra dữ liệu đầu vào không được null hoặc rỗng trước khi cập nhật.

Đoạn 2: Kế thừa trung gian `KhachHang`

```
public class KhachHang extends Nguoi {
    private String LoaiKH;

    public KhachHang() {
        super("", "");
        this.LoaiKH = "Thuong";
    }

    public KhachHang(String name, String numberPhone, String LoaiKH) {
        super(name, numberPhone);
        this.LoaiKH = (LoaiKH != null) ? LoaiKH : "Thuong";
    }

    public String getLoaiKH() { return LoaiKH; }
    public void setLoaiKH(String LoaiKH) { this.LoaiKH = LoaiKH; }
}
```

- **Giải thích từng dòng code:**
- `public class KhachHang extends Nguoi` : Quan hệ kế thừa. `KhachHang` kế thừa họ tên và số điện thoại từ `Nguoi` .
- `super(name, numberPhone)` : Gọi constructor của lớp cha `Nguoi` để gán giá trị cho `name` và `numberPhone` .
- `private String LoaiKH` : Thuộc tính phân loại khách hàng (Thường, VIP, Thân thiết). Mặc định nếu không truyền là `"Thuong"` .

Đoạn 3: Interface `IGiamGiaKhachHang`

```
public interface IGiamGiaKhachHang {
    double applyDiscount(double totalPrice);
}
```

- **Giải thích từng dòng code:**
- Định nghĩa một hợp đồng giao tiếp (Contract). Bất kỳ đối tượng nào tham gia vào hệ thống thanh toán có chính sách ưu đãi giảm giá đều phải hiện thực hàm `applyDiscount`.
- Đầu vào là `totalPrice` (tổng tiền trước giảm), đầu ra là tổng tiền sau khi đã trừ đi số tiền được chiết khấu.

Đoạn 4: Khách hàng VIP `KhachHangVIP` giảm 20%

```
public class KhachHangVIP extends KhachHang implements IGiamGiaKhachHang {
    private String cardCodeVIP;

    public KhachHangVIP(String name, String numberPhone, String cardCodeVIP) {
        super(name, numberPhone, "VIP");
        this.cardCodeVIP = (cardCodeVIP != null) ? cardCodeVIP.trim().replace(",", " ") : "";
    }

    public String getCardCodeVIP() { return cardCodeVIP; }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.2);
    }
}
```

- **Giải thích từng dòng code:**
- `extends KhachHang implements IGiamGiaKhachHang` : Kế thừa thuộc tính khách hàng và cam kết cài đặt chiết khấu.
- `private String cardCodeVIP` : Thuộc tính riêng của khách VIP là mã số thẻ định danh VIP.
- `@Override public double applyDiscount(...)` : Ghi đè phương thức từ interface. Công thức: `totalPrice - (totalPrice * 0.2)` -> Giảm trực tiếp **20%** tổng giá trị hóa đơn.

Đoạn 5: Khách hàng Thân thiết `KhachHangThanThiet` giảm 10% & Tích điểm

```
public class KhachHangThanThiet extends KhachHang implements IGiamGiaKhachHang {
    private int diemTichLuy;

    public KhachHangThanThiet(String name, String numberPhone, int diemTichLuy) {
        super(name, numberPhone, "ThanThiet");
        this.diemTichLuy = (diemTichLuy >= 0) ? diemTichLuy : 0;
    }

    public int getDiemTichLuy() { return diemTichLuy; }
    public void setDiemTichLuy(int diemTichLuy) {
        if (diemTichLuy >= 0) this.diemTichLuy = diemTichLuy;
    }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.10);
    }
}
```

- **Giải thích từng dòng code:**

- `private int diemTichLuy` : Quản lý điểm thưởng tích lũy của khách hàng thân thiết. Ràng buộc điểm không được âm (`diemTichLuy >= 0`).
- `@Override public double applyDiscount(...)` : Chiết khấu ưu đãi **10%** tổng hóa đơn cho khách thân thiết.

Đoạn 6: Nhánh Nhân viên `NhanVien` & `NhanVienQuanLy`

```
public class NhanVien extends Nguoi implements IGiamGiaKhachHang {
    private String staffID;

    public NhanVien(String name, String numberPhone, String staffID) {
        super(name, numberPhone);
        this.staffID = (staffID != null) ? staffID.trim().replace(",", " ") : "";
    }

    @Override
    public double applyDiscount(double totalPrice) {
        return totalPrice - (totalPrice * 0.5);
    }
}
```

```
public class NhanVienQuanLy extends NhanVien {
    private double phuCapTrachNhiem;

    public NhanVienQuanLy(String name, String numberPhone, String staffID, double phuCapTrachNhiem)
{
        super(name, numberPhone, staffID);
        this.phuCapTrachNhiem = (phuCapTrachNhiem >= 0) ? phuCapTrachNhiem : 0;
    }

    public double getPhuCapTrachNhiem() { return phuCapTrachNhiem; }
}
```

- **Giải thích từng dòng code:**
- `NhanVien` kế thừa từ `Nguoi` và cài đặt `IGiamGiaKhachHang` .
- Mức giảm giá nội bộ dành cho nhân viên mua sắm là **50%** (`totalPrice * 0.5`).
- `NhanVienQuanLy` kế thừa từ `NhanVien` : Quản lý cũng là một nhân viên, do đó tự động được hưởng mức giảm giá 50% từ lớp cha.
- `private double phuCapTrachNhiem` : Thuộc tính riêng mở rộng của Quản lý, quản lý khoản tiền phụ cấp chức vụ hàng tháng.

Đoạn 7: Lớp độc lập `NhaCungCap`

```
public class NhaCungCap {
    private String maNCC;
    private String tenNCC;
    private String diaChi;
    private String soDienThoai;
```

- **Giải thích:** Quản lý danh mục đối tác may mặc cung ứng hàng hóa cho shop thời trang. Đóng gói đầy đủ getter/setter và phương thức in thông tin chuẩn hàng ngang (`hienThiThongTin()`), phục vụ trực tiếp cho Menu 9 trong hệ thống.

3. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU & CÁCH TRẢ LỜI ĐẠT 10 ĐIỂM

CÂU HỎI VẤN ĐÁP: Tại sao cả `KhachHangVIP` và `NhanVien` đều cài đặt interface `IGiamGiaKhachHang`? Tại sao không viết phương thức này trong lớp `Nguoi`?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, nếu đặt phương thức giảm giá ở lớp `Nguoi`, thì mọi người bình thường bước vào shop đều có quyền giảm giá, điều này trái với thực tế kinh doanh. Khách hàng vãng lai thông thường không được giảm giá. Vì vậy, nhóm sử dụng Interface `IGiamGiaKhachHang` để đại diện cho một "năng lực đặc biệt" (Capability). Chỉ những đối tượng thuộc diện ưu đãi như VIP (20%), Thân thiết (10%) hoặc Nhân viên nội bộ (50%) mới được gắn interface này.

CÂU HỎI VẤN ĐÁP: Trong lớp `HoaDon`, khi tính tổng tiền, cơ chế đa hình chiết khấu diễn ra như thế nào?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, trong `HoaDon`, nhóm lưu trữ một thuộc tính kiểu cha: `private Nguoi person`. Khi tính tiền:

```java
if (person instanceof IGiamGiaKhachHang) {
total = ((IGiamGiaKhachHang) person).applyDiscount(total);
}
```

Tại thời điểm chạy (Runtime), máy ảo Java sẽ kiểm tra xem đối tượng thực tế mà con trỏ `person` đang trỏ tới là gì. Nếu là `KhachHangVIP`, nó tự động kích hoạt hàm giảm 20%; nếu là `NhanVien`, nó giảm 50%. Nhờ đa hình, code hoàn toàn không cần các khối lệnh `if-else` kiểm tra từng loại lớp, giúp code ngắn gọn và dễ mở rộng.

CÂU HỎI VẤN ĐÁP: Tại sao `NhanVienQuanLy` không cần viết lại phương thức `applyDiscount` mà vẫn được giảm giá 50%?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, đó là nhờ **Tính kế thừa (Inheritance)**. `NhanVienQuanLy` kế thừa từ `NhanVien`, mà `NhanVien` đã hiện thực `IGiamGiaKhachHang` và định nghĩa `applyDiscount` giảm 50%. Do lớp Quản lý không ghi đè lại phương thức này, Java sẽ tự động sử dụng phương thức được thừa hưởng từ lớp cha.

HƯỚNG DẪN HỌC & BẢO VỆ VẤN ĐÁP ĐỒ ÁN OOP (ĐIỂM 10/10)

> **Phân công thành viên:** Thành viên 3

> **Các lớp phụ trách (4 lớp & 3 file dữ liệu):** `DanhSachSanPham` , `DanhSachKhachHang` , `ChiTietHoaDon` , `HoaDon` , `products.txt` , `users.txt` , `hoadon.txt` .

> **Nhiệm vụ cốt lõi:** Nắm vững cấu trúc Lớp mảng danh sách đối tượng (Class Array), cơ chế tự sinh mã duy nhất, kỹ thuật Đọc/Ghi file text chuẩn UTF-8 chống lỗi font tiếng Việt và ý nghĩa các thành phần Static.

1. MÔ HÌNH LỚP MẢNG & DÒNG CHẢY DỮ LIỆU FILE

```
[products.txt]  <--->  [DanhSachSanPham]  <--->  [QuanLyCuaHang]
                                |                       |
                          (quản lý)                (quản lý)
                                v                       v
                    ArrayList<SanPhamThoiTrang>       [HoaDon]
                                                        | (chứa)
   [users.txt]     <--->  [DanhSachKhachHang]           v
                                |                 ArrayList<ChiTietHoaDon>
                          (quản lý)                     |
                                v                 (lưu vết)
                          ArrayList<Nguoi>              v
                                                  [hoadon.txt]
```

- **Lớp mảng (Class Array):** Đóng gói bộ sưu tập `ArrayList` bên trong class, không để lộ cấu trúc dữ liệu ra ngoài, cung cấp các hàm nghiệp vụ chuẩn: Thêm, Xóa, Sửa, Tìm kiếm, Tự sinh mã.
- **Lưu trữ dữ liệu:** Đọc/Ghi đồng bộ dữ liệu kho hàng và tài khoản bằng luồng `UTF-8` .

2. GIẢI THÍCH CHI TIẾT TỪNG ĐOẠN CODE CỐT LÕI

Đoạn 1: Thuật toán Tự sinh mã ID tự động (`taoMaTuDong`)

```
public String taoMaTuDong(String type) {
        String prefix = "SP";
        if (type != null) {
            String t = type.toLowerCase();
            if (t.equals("aokhoac")) prefix = "AK";
            else if (t.equals("quanjean") || t.equals("jeans")) prefix = "QJ";
            else if (t.equals("aothun") || t.equals("tshirt")) prefix = "AT";
            else if (t.equals("phukien") || t.equals("accessory")) prefix = "PK";
            else if (t.equals("vaydam")) prefix = "VD";
            else if (t.equals("giaydep")) prefix = "GD";
        }

        int maxId = 0;
        for (SanPhamThoiTrang p : danhSach) {
            if (p.getMaSP() != null && p.getMaSP().toUpperCase().startsWith(prefix)) {
                String numPart = p.getMaSP().substring(prefix.length());
                try {
                    int id = Integer.parseInt(numPart);
                    if (id > maxId) maxId = id;
                } catch (NumberFormatException ignored) {}
            }
        }
        return String.format("%s%03d", prefix, maxId + 1);
```

```
    }
```

- **Giải thích từng dòng code:**
- `String prefix` : Xác định tiền tố theo loại sản phẩm (`AK` cho Áo khoác, `QJ` cho Quần jean, `GD` cho Giày dép...).
- Vòng lặp `for (SanPhamThoiTrang p : danhSach)` : Quét toàn bộ danh sách sản phẩm hiện có trong kho.
- `p.getMaSP().toUpperCase().startsWith(prefix)` : Lọc những sản phẩm có cùng tiền tố.
- `p.getMaSP().substring(prefix.length())` : Cắt bỏ phần chữ cái để lấy phần số thứ tự phía sau. Ví dụ `"AK005".substring(2)` lấy được `"005"` .
- `Integer.parseInt(numPart)` : Chuyển chuỗi số thành số nguyên và tìm giá trị lớn nhất `maxId` .
- `String.format("%s%03d", prefix, maxId + 1)` : Sinh mã mới bằng tiền tố kết hợp số tiếp theo được đệm 3 chữ số (VD: `AK001` , `AK002` ...). Người dùng không cần tự gõ mã bằng tay, tránh 100% nguy cơ trùng lặp mã sản phẩm.

Đoạn 2: Thao tác Thêm, Sửa, Xóa trong Lớp mảng `DanhSachSanPham`

```
public boolean them(SanPhamThoiTrang sp) {
        if (sp == null || sp.getMaSP() == null || sp.getMaSP().trim().isEmpty()) {
            return false;
        }
        if (timKiem(sp.getMaSP()) != null) {
            System.out.println("San pham co ma " + sp.getMaSP() + " da ton tai!");
            return false;
        }
        danhSach.add(sp);
        return true;
    }

    public boolean xoa(String maSP) {
        if (maSP == null) return false;
        for (int i = 0; i < danhSach.size(); i++) {
            if (danhSach.get(i).getMaSP() != null &&
danhSach.get(i).getMaSP().equalsIgnoreCase(maSP.trim())) {
                danhSach.remove(i);
                return true;
            }
        }
        return false;
    }

    public boolean sua(String maSP, String tenMoi, double giaMoi) {
        SanPhamThoiTrang sp = timKiem(maSP);
        if (sp == null) return false;
        if (tenMoi != null && !tenMoi.trim().isEmpty()) sp.setTenSP(tenMoi.trim());
        if (giaMoi >= 0) sp.setGiaBan(giaMoi);
        return true;
    }
```

- **Giải thích từng dòng code:**
- `them(...)` : Kiểm tra mã sản phẩm đã tồn tại hay chưa bằng hàm `timKiem` . Nếu đã có thì từ chối thêm (tránh trùng khóa chính). Trả về `boolean` thể hiện trạng thái.

- `xoa(...)` : Dùng vòng lặp tìm kiếm đối tượng có mã khớp không phân biệt hoa thường (`equalsIgnoreCase`). Khi tìm thấy, gọi `danhSach.remove(i)` và dừng lặp ngay.
- `sua(...)` : Tìm đối tượng theo mã, sau đó cập nhật tên mới và giá bán mới nếu dữ liệu hợp lệ.

Đoạn 3: Kỹ thuật Đọc/Ghi File Text chuẩn UTF-8

```
public void docTuFile(String fileName) {
        File file = new File(fileName);
        if (!file.exists()) return;

        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8))) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.trim().isEmpty()) continue;
                String[] parts = line.split(",");
                if (parts.length < 5) continue;

                String type = parts[0].trim();
                String code = parts[1].trim();
                String name = parts[2].trim();
                double price = Double.parseDouble(parts[3].trim());

                switch (type.toLowerCase()) {
                    case "aokhoac":
                        boolean haveHat = Boolean.parseBoolean(parts[4].trim());
                        them(new AoKhoac("AoKhoac", code, name, price, haveHat));
                        break;
                    case "giaydep":
                        int szGiay = Integer.parseInt(parts[4].trim());
                        them(new GiayDep("GiayDep", code, name, price, szGiay));
                        break;
                    // ... các loại sản phẩm khác
                }
            }
        } catch (IOException e) {
            System.out.println("Da xay ra loi khi doc du lieu tu file.");
        }
    }
```

- **Giải thích từng dòng code:**
- `try (...)` : Cú pháp **Try-with-resources** (từ Java 7 trở lên). Đảm bảo luồng đọc file `reader` luôn được tự động đóng (`close()`) ngay cả khi có ngoại lệ xảy ra, tránh rò rỉ bộ nhớ (Memory Leak) hoặc khóa file.
- `new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8)` : Chỉ định tường minh bảng mã **UTF-8**. Giúp đọc chính xác tiếng Việt có dấu ("Áo gió ấm", "Váy dạ hội") trên mọi môi trường Windows mà không bị biến thành ký tự lạ (`??`).
- `line.split(",")` : Tách từng dòng văn bản thành mảng các chuỗi con dựa trên dấu phẩy.
- Cấu trúc `switch-case` : Phân tích cột đầu tiên (`type`) để khởi tạo chính xác đối tượng lớp con tương ứng (Kỹ thuật Factory Pattern).

Đoạn 4: Ghi dữ liệu xuống File Text

```
public void ghiVaoFile(String fileName) {
        try (BufferedWriter writer = new BufferedWriter(
```

```
                new OutputStreamWriter(new FileOutputStream(fileName), StandardCharsets.UTF_8))) {
            for (SanPhamThoiTrang sp : danhSach) {
                writer.write(sp.getStringSaveFile());
                writer.newLine();
            }
            writer.flush();
        } catch (IOException e) {
            System.out.println("Da xay ra loi khi luu du lieu vao file.");
        }
    }
```

- **Giải thích từng dòng code:**
- `BufferedWriter` và `OutputStreamWriter` kết hợp `StandardCharsets.UTF_8` : Ghi dữ liệu dạng luồng đệm tốc độ cao và chuẩn font tiếng Việt.
- `sp.getStringSaveFile()` : Gọi phương thức đa hình của từng đối tượng sản phẩm để lấy chuỗi định dạng CSV tương ứng.
- `writer.newLine()` : Xuống dòng tương thích với hệ điều hành đang chạy (CRLF trên Windows).
- `writer.flush()` : Ép toàn bộ dữ liệu còn lưu trong bộ nhớ đệm ghi ngay xuống đĩa cứng.

Đoạn 5: Chi Tiết Hóa Đơn (`ChiTietHoaDon`) & Quản lý Hóa đơn (`HoaDon`)

```
public class ChiTietHoaDon {
    private SanPhamThoiTrang sanPham;
    private int soLuong;

    public void tangSoLuong(int amount) {
        if (amount > 0) this.soLuong += amount;
    }

    public double tinhThanhTienSauGiam(double phanTramGiamGia) {
        if (sanPham == null) return 0;
        double giaSauGiam1SP = sanPham.calculateAfterDiscount(phanTramGiamGia);
        return (giaSauGiam1SP + sanPham.VATcalculation()) * soLuong;
    }
}
```

- **Giải thích từng dòng code:**
- Biểu diễn quan hệ chứa đựng (**Aggregation**): Một Hóa đơn gồm nhiều Chi tiết hóa đơn; mỗi Chi tiết hóa đơn giữ tham chiếu đến một `SanPhamThoiTrang` và số lượng mua `soLuong` .
- `tangSoLuong` : Khi khách hàng nhập trùng mã sản phẩm đã có trong giỏ hàng, hệ thống không tạo dòng mới mà cộng dồn số lượng.
- `tinhThanhTienSauGiam` : Tính tổng tiền cho dòng hàng bao gồm: `(Đơn giá sau giảm + Thuế VAT) * Số lượng` .

Đoạn 6: Các Thành phần Static (Biến Static & Hàm Static)

```
public static final double TI_LE_VAT = 0.10;
private static int tongSoLuongSanPhamDaTao = 0;
private static int tongSoHoaDon = 0;

public static String formatTien(double amount) {
    DecimalFormatSymbols symbols = new DecimalFormatSymbols();
    symbols.setGroupingSeparator('.');
```

```
    DecimalFormat df = new DecimalFormat("#,##0", symbols);
    return df.format(Math.round(amount)) + " VND";
}
```

- **Giải thích từng dòng code:**
- `public static final double TI_LE_VAT = 0.10` : Hằng số tĩnh đại diện cho mức thuế VAT 10%. Được truy xuất trực tiếp qua tên lớp: `SanPhamThoiTrang.TI_LE_VAT` mà không cần khởi tạo đối tượng.
- `tongSoLuongSanPhamDaTao` & `tongSoHoaDon` : Biến đếm mức lớp (Class-level variable). Cả chương trình chỉ tồn tại đúng 1 vùng nhớ duy nhất chia sẻ cho mọi thực thể để thống kê số lượng.
- `formatTien(double)` : Phương thức tĩnh tiện ích. Chuyển đổi số thực sang định dạng tiền tệ Việt Nam có dấu chấm phân cách hàng nghìn (ví dụ: `1800000 -> 1.800.000 VND`).

3. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU & CÁCH TRẢ LỜI ĐẠT 10 ĐIỂM

CÂU HỎI VẤN ĐÁP: Tại sao phải sử dụng `StandardCharsets.UTF_8` khi đọc và ghi file? Nếu chỉ dùng `new FileReader("products.txt")` thì có vấn đề gì?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, nếu chỉ dùng `new FileReader(file)` thì Java sẽ sử dụng bảng mã mặc định của hệ điều hành máy khách (trên Windows tiếng Việt thường là Windows-1258 hoặc Windows-1252). Khi đó, các tên sản phẩm có dấu tiếng Việt như "Áo khoác gió", "Váy dạ hội" sẽ bị biến thành các ký tự rác (lỗi font). Việc dùng `InputStreamReader` và `OutputStreamWriter` kết hợp `StandardCharsets.UTF_8` ép buộc mọi thao tác đọc/ghi phải dùng chuẩn quốc tế UTF-8, đảm bảo hiển thị hoàn hảo 100% tiếng Việt trên mọi hệ điều hành.

CÂU HỎI VẤN ĐÁP: Trong khối lệnh đọc file, cú pháp `try (BufferedReader reader = ...)` có ý nghĩa gì đặc biệt?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, đây là cấu trúc **Try-with-resources** của Java. Mọi đối tượng implements interface `AutoCloseable` được khai báo trong ngoặc đơn của `try` sẽ được hệ thống tự động đóng tài nguyên (gọi hàm `close()`) sau khi khối lệnh kết thúc, kể cả khi xảy ra Exception. Cách viết này an toàn hơn khối `try-catch-finally` truyền thống, giúp mã nguồn gọn gàng và ngăn ngừa rò rỉ tài nguyên luồng file.

CÂU HỎI VẤN ĐÁP: Thuộc tính static khác với thuộc tính thông thường (instance variable) như thế nào?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy:

• Thuộc tính thông thường thuộc về từng đối tượng cụ thể. Mỗi khi `new` một đối tượng, một vùng nhớ mới được cấp phát riêng biệt.

• Thuộc tính static thuộc về lớp. Nó chỉ được cấp phát một vùng nhớ duy nhất trong bộ nhớ Heap/Metaspace khi nạp lớp, và được chia sẻ chung cho tất cả các đối tượng của lớp đó. Trong đồ án, biến `TI_LE_VAT` và `tongSoHoaDon` là static vì chúng mang giá trị dùng chung toàn cục cho cửa hàng.

HƯỚNG DẪN HỌC & BẢO VỆ VẤN ĐÁP ĐỒ ÁN OOP (ĐIỂM 10/10)

> **Phân công thành viên:** Thành viên 4 (Trưởng nhóm / Điều phối Demo chính)

> **Các lớp phụ trách (3 lớp):** `QuanLyCuaHang` , `Main` , `TestCases` .

> **Nhiệm vụ cốt lõi:** Nắm vững kiến trúc lớp điều phối trung tâm (Facade Pattern), toàn bộ vòng lặp giao diện Console Menu 10 chức năng (CRUD), quy trình bán hàng - thanh toán - in phiếu bảo hành và kịch bản vấn đáp toàn diện đạt điểm 10.

1. MÔ HÌNH KIẾN TRÚC ĐIỀU PHỐI HỆ THỐNG (FACADE PATTERN)

```
[Người dùng / Console]
                                    |
                                    v
                              [Main.java] (Giao diện View)
                                    |
                                    v
                          [QuanLyCuaHang.java] (Controller / Facade)
                              /     |     \
                             /      |      \
                            v       v       v
                  [DanhSachSP]  [DanhSachKH] [ArrayList<NhaCungCap>]
                        |           |                 |
                        v           v                 v
                   products.txt  users.txt     (Nhà máy may mặc)
```

- **Mô hình Facade / Controller (`QuanLyCuaHang`):** Gom toàn bộ các kho dữ liệu rời rạc (sản phẩm, khách hàng, nhà cung cấp) vào một đầu mối duy nhất. Lớp `Main` không cần tương tác trực tiếp với file hay mảng con, chỉ cần gọi qua `store` .
- **Cơ chế Tự động Đồng bộ (Auto-save):** Mọi thao tác Thêm, Sửa, Xóa đều tự động kích hoạt `saveDataIntoFile()` , đảm bảo dữ liệu luôn được ghi tức thì xuống đĩa cứng.

2. GIẢI THÍCH CHI TIẾT TỪNG ĐOẠN CODE CỐT LÕI

Đoạn 1: Lớp điều phối trung tâm `QuanLyCuaHang`

```
public class QuanLyCuaHang {
    private DanhSachSanPham dsSanPham;
    private DanhSachKhachHang dsKhachHang;
    private ArrayList<NhaCungCap> dsNhaCungCap;

    public QuanLyCuaHang() {
        this.dsSanPham = new DanhSachSanPham();
        this.dsKhachHang = new DanhSachKhachHang();
        this.dsNhaCungCap = new ArrayList<>();
        khoiTaoNhaCungCapMacDinh();
    }

    public void addProduct(SanPhamThoiTrang product) {
        if (dsSanPham.them(product)) {
            saveDataIntoFile();
        }
    }
```

```
    public void removeProduct(String MaSP) {
        if (dsSanPham.xoa(MaSP)) {
            saveDataIntoFile();
            System.out.println("Da xoa san pham co ma: " + MaSP);
        } else {
            System.out.println("Khong tim thay san pham co ma " + MaSP + ".");
        }
    }

    public void saveDataIntoFile() {
        dsSanPham.ghiVaoFile("products.txt");
        dsKhachHang.ghiVaoFile("users.txt");
    }

    public void loadDataFromFile(String fileName) {
        dsSanPham.docTuFile(fileName);
        dsKhachHang.docTuFile("users.txt");
    }
}
```

- **Giải thích từng dòng code:**
- `private DanhSachSanPham dsSanPham; private DanhSachKhachHang dsKhachHang;` : Đóng gói các lớp mảng đối tượng thành các thuộc tính riêng biệt của hệ thống cửa hàng.
- `khoiTaoNhaCungCapMacDinh()` : Nạp sẵn danh mục đối tác may mặc lớn (Việt Tiến, An Phước, Biti's) ngay khi khởi động.
- `addProduct(...)` & `removeProduct(...)` : Kiểm tra kết quả thực hiện từ lớp danh sách; nếu thành công (`true`) thì lập tức gọi `saveDataIntoFile()` để đồng bộ file ngay. Người dùng không sợ mất dữ liệu khi mất điện hay tắt máy đột ngột.
- `loadDataFromFile(...)` : Đọc toàn bộ kho hàng và danh sách người dùng khi mở ứng dụng.

Đoạn 2: Vòng lặp Menu chính trong `Main.java`

```
public static void main(String[] args) {
        QuanLyCuaHang store = new QuanLyCuaHang();
        store.loadDataFromFile("products.txt");
        Scanner scanner = new Scanner(System.in);

        while (true) {
            inMenu();
            System.out.print("Nhap lua chon cua ban: ");
            String input = scanner.nextLine().trim();
            int choice = -1;
            try {
                choice = Integer.parseInt(input);
            } catch (NumberFormatException e) {
                System.out.println("Vui long nhap so hop le!");
                continue;
            }

            switch (choice) {
                case 1: // Them san pham
                case 2: // Xoa san pham
                case 3: // Tim kiem san pham (theo ma hoac theo ten)
                case 4: // Sua san pham (ten, gia, thuoc tinh rieng)
                case 5: // Hien thi tat ca san pham
                case 6: // Xem khuyen mai 10%
                case 7: // Quan ly Khach hang / Nhan vien (CRUD)
```

```
                case 8: // Thanh toan hoa don & In bao hanh
                case 9: // Xem danh sach Nha cung cap
                case 0:
                    store.saveDataIntoFile();
                    System.out.println("Da luu du lieu an toan. Thoat chuong trinh.");
                    return;
            }
        }
    }
```

- **Giải thích từng dòng code:**
- Bọc trong vòng lặp vô hạn `while (true)` : Đảm bảo chương trình chạy liên tục cho đến khi người dùng chủ động chọn `0. Luu va thoat` .
- Đọc dữ liệu bằng `scanner.nextLine().trim()` kết hợp `Integer.parseInt` : Tránh triệt để lỗi "trôi lệnh" (Skip line) khét tiếng của `scanner.nextInt()` trong Java khi nhấn Enter.
- Khối `try-catch (NumberFormatException)` : Ngăn chặn chương trình bị crash khi người dùng vô tình nhập chữ cái thay vì số.

Đoạn 3: Menu 4 - Chức năng Sửa toàn diện (Tên, Giá, Thuộc tính riêng)

```
if (editOpt.equals("4")) {
        if (oldSp instanceof AoKhoac) {
            System.out.println("Ao co non khong? (1: Co / 0: Khong): ");
            boolean hasHat = scanner.nextLine().trim().equals("1");
            ((AoKhoac) oldSp).setHasHat(hasHat);
        } else if (oldSp instanceof QuanJean) {
            System.out.println("Nhap kieu dang moi: ");
            String st = scanner.nextLine().trim().replace(",", " ");
            ((QuanJean) oldSp).setStyle(st);
        } else if (oldSp instanceof AoThun) {
            System.out.println("Nhap kich co moi (S/M/L/XL): ");
            String sz = scanner.nextLine().trim();
            ((AoThun) oldSp).setSize(sz);
        } else if (oldSp instanceof GiayDep) {
            System.out.println("Nhap size giay moi (35 - 45): ");
            int sg = Integer.parseInt(scanner.nextLine().trim());
            ((GiayDep) oldSp).setSizeGiay(sg);
        }
        store.saveDataIntoFile();
    }
```

- **Giải thích từng dòng code:**
- `oldSp instanceof AoKhoac` : Toán tử kiểm tra kiểu thực tế của đối tượng tại runtime.
- `((AoKhoac) oldSp)` : Kỹ thuật **Downcasting (Ép kiểu xuống)** từ lớp cha `SanPhamThoiTrang` về đúng lớp con cụ thể để gọi được các phương thức riêng biệt (`setHasHat` , `setSize` , `setStyle` , `setSizeGiay`).
- Cập nhật xong gọi ngay `store.saveDataIntoFile()` .

Đoạn 4: Menu 8 - Quy trình Thanh toán Hóa đơn & In Phiếu bảo hành

```
HoaDon newbill = new HoaDon(newBillCode, null);
    // Vòng lặp cho phép khách chọn nhiều sản phẩm và số lượng...
    SanPhamThoiTrang product = store.getProductByCode(pCode);
    newbill.addProduct(product, sl);
```

```
    // Nhập số điện thoại tra cứu đối tượng:
    Nguoi person = store.getVIPOrStaffByPhone(sdt);
    newbill.setPerson(person);

    // In hóa đơn ra màn hình và lưu file:
    newbill.displayBill();
    newbill.xuatHoaDonRaFile("hoadon.txt");
```

```
// Bên trong HoaDon.java:
    public void printAllWarrantyCards() {
        for (ChiTietHoaDon ct : chiTietList) {
            SanPhamThoiTrang sp = ct.getSanPham();
            if (sp instanceof IBaoHanh) {
                ((IBaoHanh) sp).printWarrantyCard();
            }
        }
    }
```

- **Giải thích từng dòng code:**
- `newbill.setPerson(person)` : Gắn đối tượng khách hàng vào hóa đơn. Nếu là Khách VIP tự động chiết khấu 20%, nếu là Khách thân thiết giảm 10%, nếu là Nhân viên giảm 50%.
- `sp instanceof IBaoHanh` : Duyệt danh sách các món hàng đã mua trong đơn; chỉ những món hàng cài đặt interface bảo hành mới gọi đa hình `printWarrantyCard()` .
- `newbill.xuatHoaDonRaFile("hoadon.txt")` : Tự động append lịch sử hóa đơn hoàn chỉnh vào file văn bản để phục vụ đối soát kế toán.

3. KỊCH BẢN DEMO ĐỒ ÁN TRƯỚC GIẢNG VIÊN (7 BƯỚC ĐẠT ĐIỂM 10)

6. **Bước 1 (Giới thiệu cấu trúc - 30 giây):** Thưa Thầy, đồ án nhóm em có tổng cộng 20 lớp đối tượng và 3 interface, tổ chức theo mô hình phân tầng: Model Sản phẩm, Model Con người, Lớp mảng quản lý, Lớp điều phối Facade và View Console.
7. **Bước 2 (Xem danh sách sản phẩm - Phím 5):** Mở bảng sản phẩm. Nhấn mạnh việc đóng khung lưới như Excel và định dạng tiền tệ Việt Nam chuẩn dấu chấm (`1.800.000 VND`).
8. **Bước 3 (Thêm sản phẩm tự sinh mã - Phím 1):** Thêm một sản phẩm mới (ví dụ Giày thể thao). Cho Thầy thấy hệ thống **tự động sinh mã `GD002`** dựa trên tiền tố loại hàng và mã lớn nhất hiện có.
9. **Bước 4 (Tìm kiếm kép - Phím 3):** Demo tìm kiếm theo mã (`AK001`) và tìm kiếm gần đúng theo từ khóa tên (`Hunter` hoặc `Polo`).
10. **Bước 5 (Sửa thuộc tính riêng - Phím 4):** Chọn sản phẩm vừa thêm và sửa thuộc tính đặc thù (sửa size giày từ 40 lên 42). Mở file `products.txt` cho Thầy thấy file được cập nhật ngay lập tức.
11. **Bước 6 (Bán hàng đa hình & In bảo hành - Phím 8):** Mua Áo khoác (`AK001` , SL: 2) và Áo thun (`AT001` , SL: 1). Nhập SĐT VIP (`0912345678`). Cho Thầy xem bảng hóa đơn tự động tính VAT 10%, giảm giá 20% cho VIP và **tự động in Phiếu bảo hành 12 tháng** cho Áo khoác. Mở file `hoadon.txt` xem nhật ký đơn hàng.
12. **Bước 7 (Minh chứng kiểm thử tự động):** Chạy `java TestCases` trên Terminal để chứng minh 73/73 testcase kiểm thử logic, đa hình và biên ngoại lệ đều PASS 100%.

4. BỘ 10 CÂU HỎI VẤN ĐÁP KINH ĐIỂN CỦA THẦY HIẾU & CÂU TRẢ LỜI ĐẠT 10 ĐIỂM

13. **Thầy hỏi: Đồ án của nhóm có đủ 15 lớp theo yêu cầu không? Kể tên?**

Trả lời: Dạ thưa Thầy, nhóm em có **20 lớp và 3 interface**. Gồm: 7 lớp sản phẩm (`SanPhamThoiTrang` , `AoKhoac` , `AoThun` , `QuanJean` , `PhuKien` , `VayDam` , `GiayDep`), 6 lớp con người (`Nguoi` , `KhachHang` , `KhachHangVIP` , `KhachHangThanThiet` , `NhanVien` , `NhanVienQuanLy`), 4 lớp quản lý (`DanhSachSanPham` , `DanhSachKhachHang` , `QuanLyCuaHang` , `NhaCungCap`), 2 lớp hóa đơn (`ChiTietHoaDon` , `HoaDon`), 1 lớp `Main` và 3 interface (`IBaoHanh` , `IGiamGia` , `IGiamGiaKhachHang`).

14. **Thầy hỏi: Tại sao lớp `SanPhamThoiTrang` lại là abstract?**

Trả lời: Vì trong thực tế không có sản phẩm nào chung chung là "Sản phẩm thời trang". Lớp cha là abstract để đóng vai trò khuôn mẫu chung, ngăn chặn việc tạo đối tượng tùy tiện bằng `new` , và bắt buộc các lớp con phải override `getThongTinRieng()` và `getStringSaveFile()` .

15. **Thầy hỏi: Đa hình được thể hiện ở đâu trong mã nguồn?**

Trả lời: Thể hiện ở 3 vị trí then chốt: (1) Gọi phương thức trừu tượng qua tham chiếu lớp cha `SanPhamThoiTrang` , (2) Đa hình bảo hành qua `IBaoHanh.printWarrantyCard()` , (3) Đa hình chiết khấu đối tượng qua `IGiamGiaKhachHang.applyDiscount()` .

16. **Thầy hỏi: Lớp mảng danh sách đối tượng trong đồ án là gì?**

Trả lời: Nhóm đã xây dựng 2 lớp mảng chuyên biệt là `DanhSachSanPham` và `DanhSachKhachHang` . Chúng đóng gói `ArrayList` bên trong, cung cấp đầy đủ các phương thức nghiệp vụ CRUD, tự sinh mã và đọc/ghi file.

17. **Thầy hỏi: Kỹ thuật ép kiểu Downcasting được dùng ở đoạn nào?**

Trả lời: Dùng ở chức năng sửa sản phẩm và in bảo hành: `((AoKhoac) oldSp).setHasHat(hasHat)` và `((IBaoHanh) sp).printWarrantyCard()` , sau khi đã kiểm tra an toàn bằng `instanceof` .

18. **Thầy hỏi: Nhóm xử lý lỗi font tiếng Việt khi đọc/ghi file văn bản như thế nào?**

Trả lời: Nhóm sử dụng `InputStreamReader` và `OutputStreamWriter` kết hợp chỉ định tường minh bảng mã `StandardCharsets.UTF_8` cùng luồng đệm `BufferedReader` / `BufferedWriter` .

19. **Thầy hỏi: Biến `tongSoHoaDon` là static có tác dụng gì?**

Trả lời: Biến static thuộc về lớp `HoaDon` , được lưu tại một vùng nhớ duy nhất và chia sẻ chung cho toàn bộ các hóa đơn sinh ra, giúp thống kê chính xác tổng số đơn hàng đã hoàn tất trong phiên làm việc.

20. **Thầy hỏi: Tại sao trong hàm tạo và setter lại có lệnh `.replace(",", " ")`?**

Trả lời: Đây là biện pháp phòng vệ dữ liệu (Data Sanitization). Vì file lưu trữ theo định dạng CSV phân tách bằng dấu phẩy, nếu tên sản phẩm hay họ tên có dấu phẩy sẽ làm vỡ cấu trúc cột khi đọc file.

21. **Thầy hỏi: Nếu có 2 sản phẩm cùng mã được quét vào giỏ hàng thì hệ thống xử lý sao?**

Trả lời: Lớp `HoaDon` sẽ duyệt giỏ hàng bằng hàm `timChiTietTheoMa(code)` . Nếu đã có, hệ thống gọi `ct.tangSoLuong(soLuongMoi)` để cộng dồn số lượng chứ không tạo thêm dòng mới, đúng với nghiệp vụ siêu thị.

22. **Thầy hỏi: Nếu ứng dụng bị mất điện tắt đột ngột thì dữ liệu có mất không?**

Trả lời: Dạ không, vì hệ thống áp dụng cơ chế tự động lưu tức thì (Auto-save). Bất kỳ thao tác thêm, xóa, sửa sản phẩm hay người dùng đều được ghi đè ngay lập tức xuống `products.txt` và `users.txt` .